Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №5
з дисципліни «Технології та методика програмування вбудованих систем»

«Дослідження моделі ЦАП»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
ст. викладач, к.т.н Орленко Сергій Петрович

Дата здачі: 30.03.2026

КИЇВ – 2026

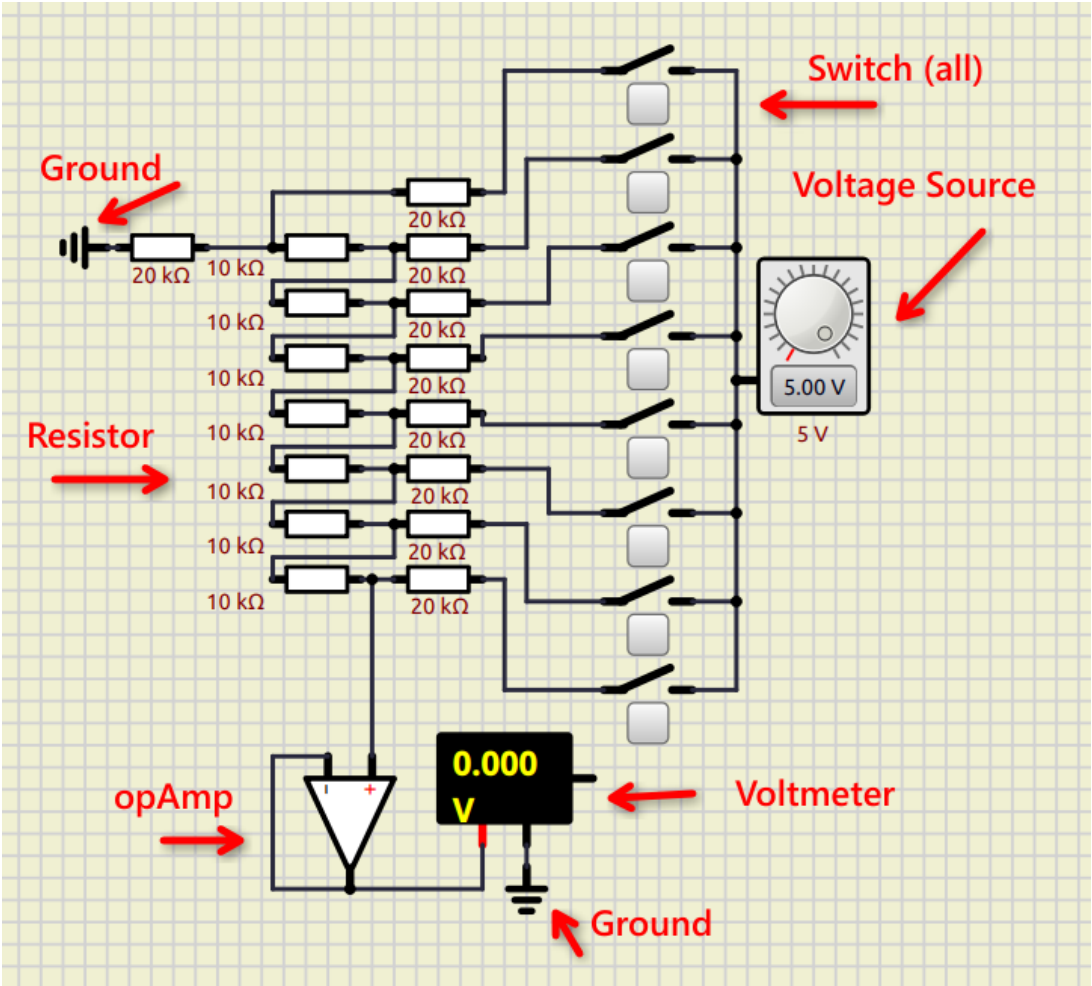
Тема. Моделювання модуля ЦАП.

Мета. Користуючись пакетом SimulIDE дослідити роботу модуля ЦАП.

Завдання. Створити модель ЦАП в пакеті SimulIDE. Розробити схему алгоритму роботи моделі та робочу програму. Створити hex-файл та підключити його до мікроконтролера. Запустити модель та виконати її дослідження згідно методичних вказівок. Зробити відповідні висновки.

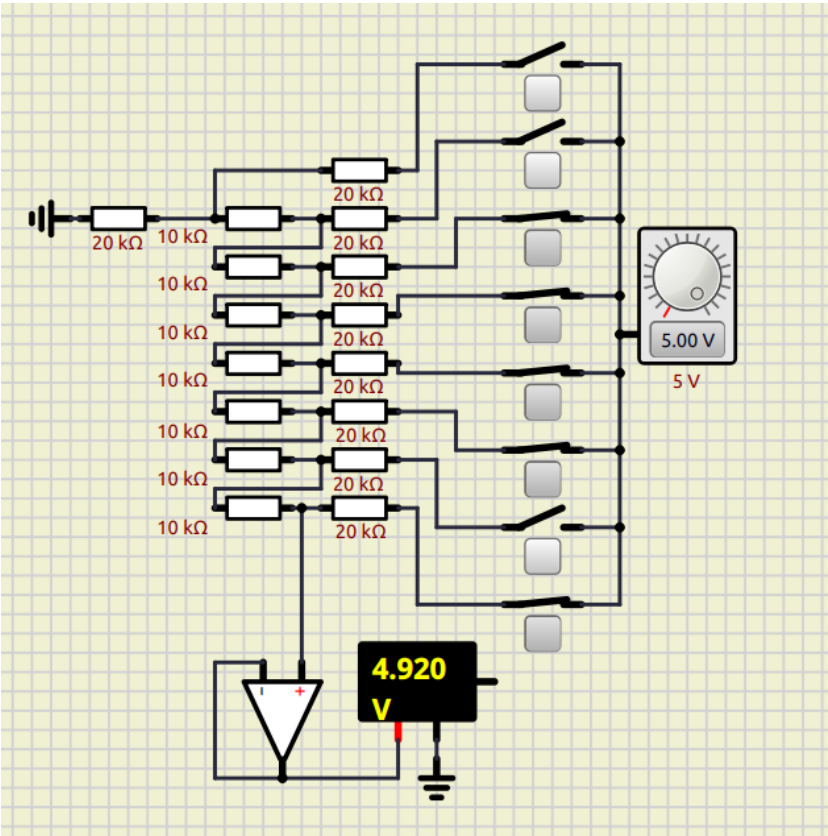
Хід роботи.

Виконання п'ятої лабораторної роботи розпочалося з базового етапу – дослідження статичної моделі цифро-аналогового перетворювача (ЦАП) без використання мікроконтролера. Ця схема необхідна для глибокого розуміння фізичних принципів роботи ЦАП на основі резисторної матриці R-2R з підсумовуванням напруг. Сама матриця складається з послідовно та паралельно з'єднаних резисторів двох номіналів: базового опору R (у нашій моделі 10 кОм) та подвійного $2R$ (20 кОм). На входи цієї матриці надходять цифрові сигнали, які відповідають значенню конкретного розряду вхідного двійкового коду. У середовищі SimulIDE роль генераторів цих логічних рівнів виконують вісім звичайних перемикачів.

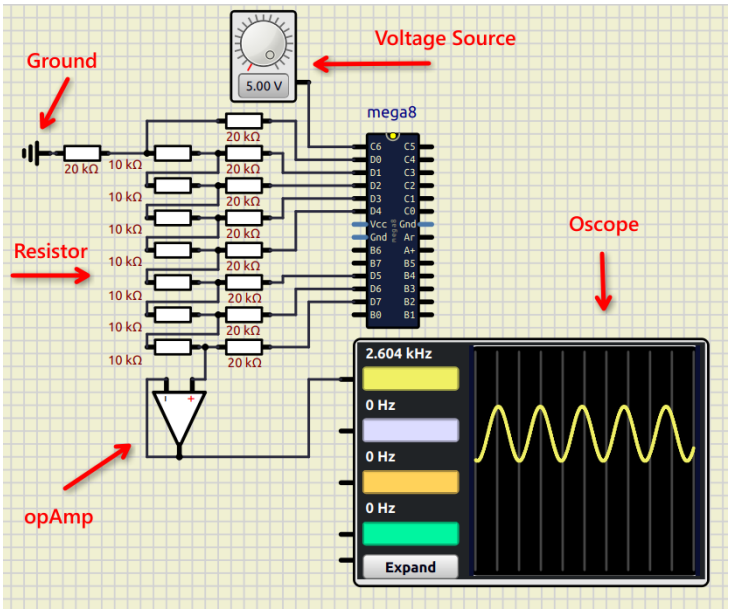


Робота резисторної матриці базується на методі суперпозиції, оскільки вона є лінійним електричним ланцюгом, і внесок у загальну вихідну напругу від кожного розряду формується абсолютно незалежно. Якщо перемикач встановлено в положення логічної одиниці, відповідний ключ перемикається у верхнє положення і гілка матриці з резистором 2R підключається до джерела опорної напруги. Якщо ж на вхід подається логічний нуль, перемикач підключає гілку матриці до спільної шини, тобто до землі. Завдяки такій конфігурації кожен наступний розряд має рівно вдвічі меншу «вагу» у формуванні вихідного сигналу, ніж попередній. Внески від усіх розрядів матриці підсумовуються і подаються на неінвертуючий вхід операційного підсилювача.

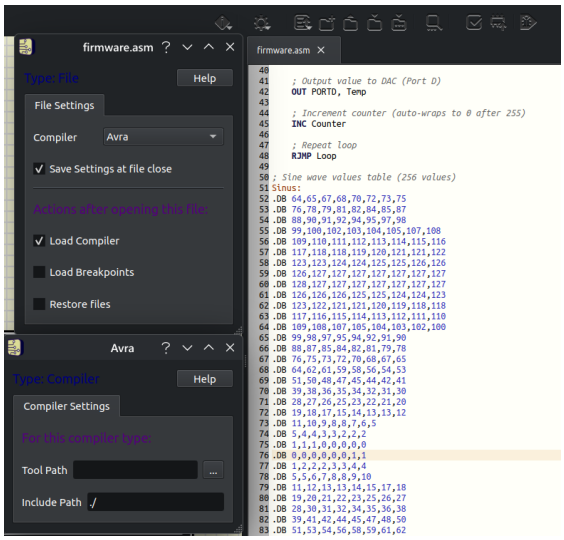
Операційний підсилювач у цій схемі використовує зворотне включення входу і виходу і працює в режимі, близькому до холостого ходу. Фактично він виконує роль повторювача напруги, коефіцієнт передачі якого дорівнює одиниці. Це необхідно для того, щоб розв'язати високоімпедансну резисторну матрицю та вимірювальні прилади, не дозволяючи навантаженню спотворити сформовану напругу. У результаті такого підсумовування на виході ЦАП ми отримуємо аналогову напругу, яка прямо пропорційна добутку значення опорного сигналу на десятковий еквівалент вхідного двійкового коду. Шляхом ручної комутації перемикачів і зняття показників вольтметра було практично підтверджено правильність розрахунків вихідної напруги для різних двійкових комбінацій.



Наступним етапом виконання лабораторної роботи став перехід від статичного ручного керування до автоматизованої динамічної генерації сигналу. Для цього логічні перемикачі були замінені на мікроконтролер ATmega8. Тактова частота ядра у властивостях компонента була встановлена на рівні 8 МГц. Вісім вхідних гілок нашої попередньо зібраної резисторної матриці R-2R були підключені до пінів порту D мікроконтролера, строго по порядку від нульового розряду PD0 до сьомого розряду PD7, який відіграє роль найстаршого значущого біта. Операційний підсилювач залишився в схемі у ролі повторювача напруги для узгодження опорів, проте замість вольтметра до його виходу був підключений щуп осцилографа для візуалізації форми швидкозмінного аналогового сигналу.



Керуюча програма для нашої схеми була реалізована мовою Асемблер. Для її збирання у середовищі SimulIDE конфігурацію компілятора було налаштовано на використання інструменту AVRA, який ідеально підходить для компіляції чистих асемблерних лістингів під архітектуру AVR.



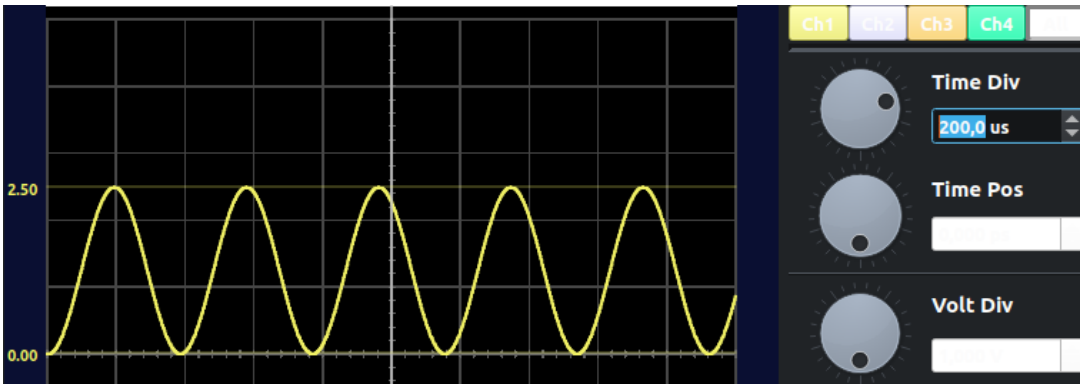
Компіляція пройшла успішно!

```
AVRA: advanced AVR macro assembler (version 1.4.2)
Pass 1...
Pass 2...
done


Assembly complete with no errors.
Segment usage:
   Code      :       146 words (292 bytes)
   Data      :         0 bytes
   EEPROM    :         0 bytes
```

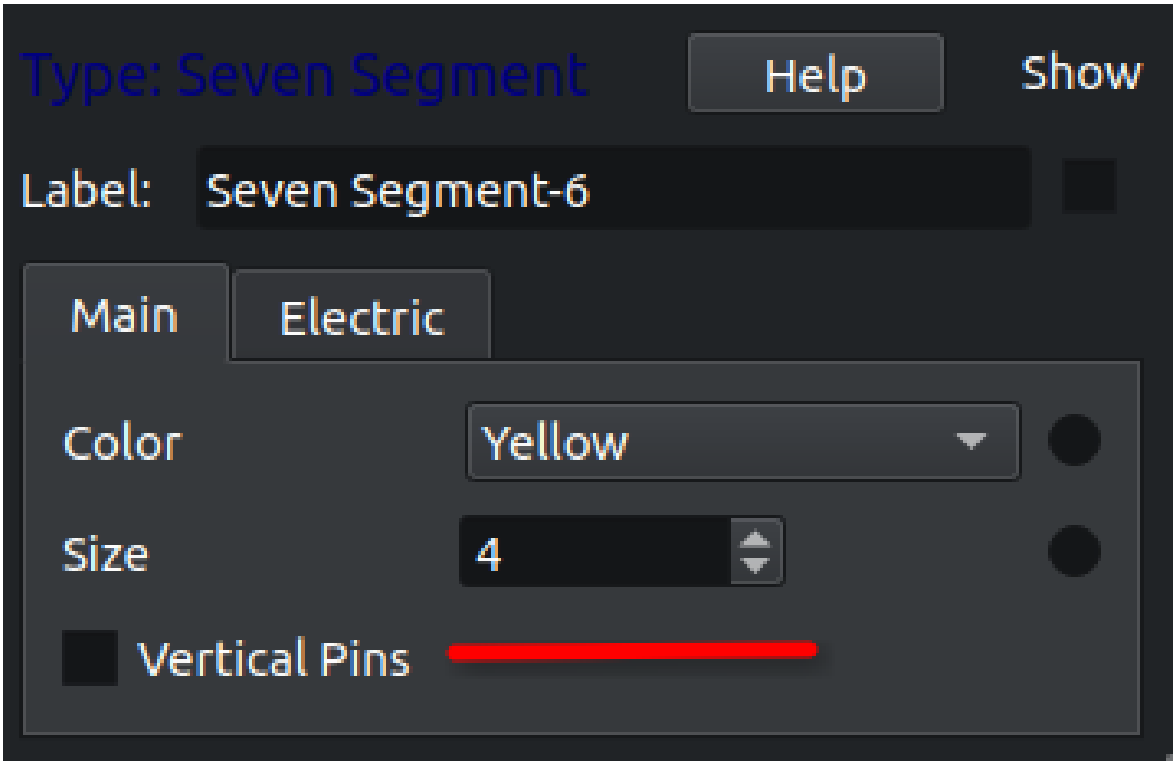
Логіка роботи програми базується на використанні заздалегідь розрахованої таблиці значень. У постійну флеш-пам'ять мікроконтролера було закладено масив із 256 байтів, які описують один повний період ідеальної синусоїди, зміщеної у додатний діапазон напруг (щоб уникнути від'ємних значень, які не підтримуються нашою однополярною схемою). Мікроконтролер у нескінченному циклі використовує регістрову пару Z як вказівник для зчитування наступного значення з цієї таблиці за допомогою команди доступу до пам'яті програм LPM. Одразу після зчитування це значення відправляється в регістр порту D командою OUT, а лічильник зміщення інкрементується. Завдяки тому, що змінна лічильника є восьмирозрядною, після досягнення значення 255 вона апаратно переповнюється і скидається в нуль, автоматично зациклюючи виведення синусоїди без необхідності використання додаткових умовних переходів.

Оскільки в коді навмисно не використовувалися жодні штучні затримки чи таймери, швидкість генерації сигналу визначалася виключно тактовою частотою процесора та сумарною кількістю тактів, необхідних для виконання базових асемблерних інструкцій циклу. Швидка синхронна зміна логічних рівнів на пінах порту D миттєво перетворювалася резисторною матрицею на відповідні аналогові рівні напруг. У результаті такого прямого синтезу на екрані віртуального осцилографа було зафіксовано чітку ступінчасту синусоїду, що повністю підтвердило правильність розрахунків та високу ефективність спроєктованого програмно-апаратного цифро-аналогового перетворювача.



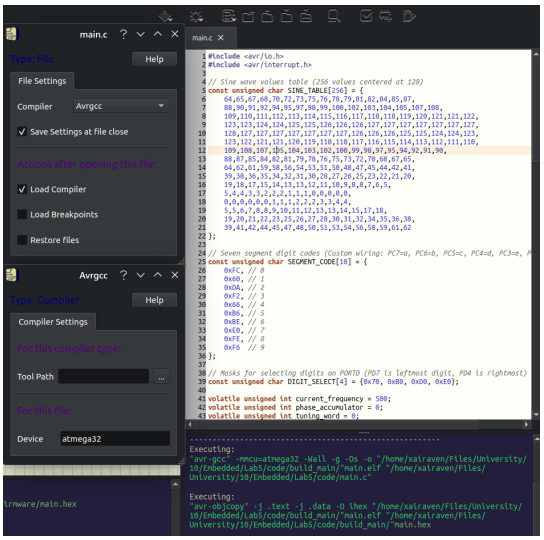
Завершальним етапом виконання лабораторної роботи стала розробка комплексного генератора синусоїдального сигналу з можливістю динамічного керування частотою. Цей етап був реалізований як повноцінна функціональна альтернатива моделі на базі мікроконтролера LPC2138, яка описана в методичних вказівках. Оскільки в програмному пакеті SimulIDE відсутня підтримка процесорів архітектури ARM із вбудованим апаратним ЦАП, систему було успішно портовано на архітектуру AVR з використанням мікроконтролера ATmega32 та готового логічного компонента 8-розрядного цифро-аналогового перетворювача.

Для формування схеми вісім вхідних ліній ЦАП були підключені до порту A мікроконтролера, аналоговий сигнал з його виходу подавався безпосередньо на осцилограф. Для візуалізації поточної частоти сигналу, як і в оригінальній схемі з методички, було застосовано цифровий дисплей. У нашому випадку це чотирирозрядний семисегментний індикатор зі спільним катодом. Керування сегментами здійснювалося через порт C, а динамічний вибір активного розряду – через старші піни порту D. Регулювання частоти сигналу було зав'язано на апаратні переривання від кнопок, підключених до пінів зовнішніх переривань INT0 та INT1.

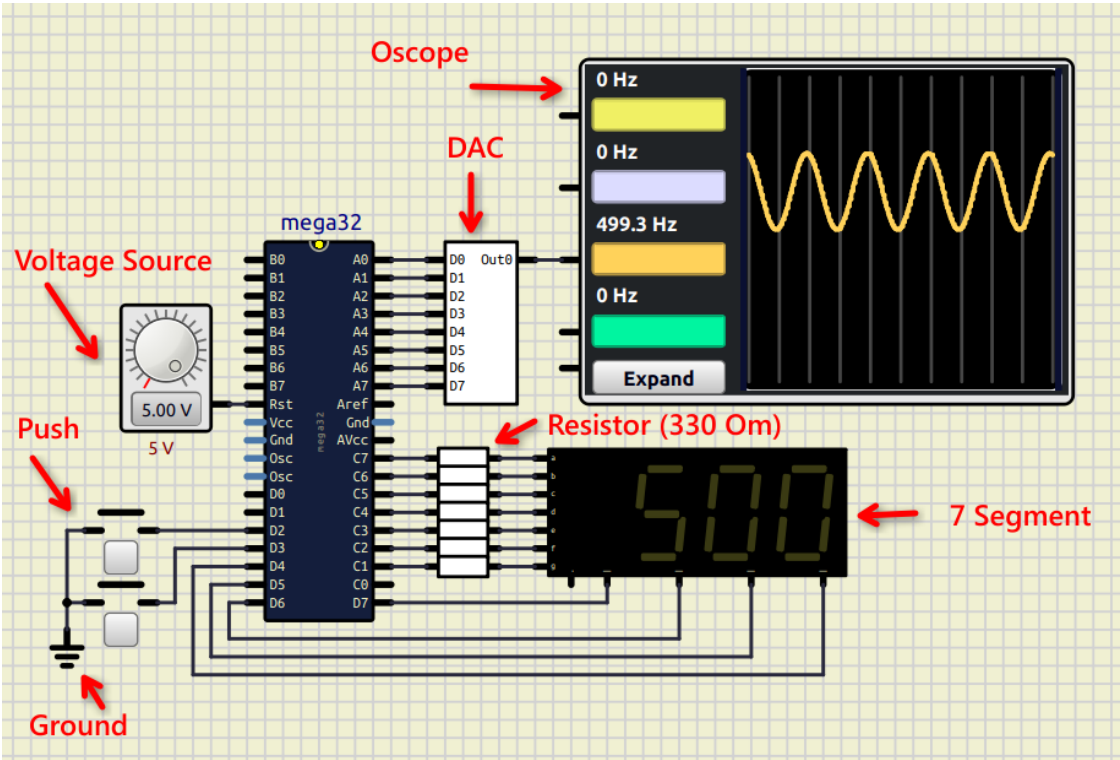


Керуюча програма для цієї складної системи була написана мовою C та скомпільована за допомогою набору інструментів `avr-gcc`. В основі програмної генерації лежить професійний метод прямого цифрового синтезу (DDS). Нульовий таймер мікроконтролера працює безперервно на максимальній швидкості без дільника, акумулюючи фазу та відправляючи відповідні значення з таблиці сину-

сів у порт A. Зміна частоти досягається не зміною швидкості таймера, а перерахунком кроку приросту фази (керуючого слова) при кожному натисканні кнопок керування.



Паралельно з генерацією сигналу, перший апаратний таймер працює в режимі скидання при збігу (CTC) з частотою 1000 Гц. Його переривання забезпечує непомітну для людського ока динамічну індикацію на семисегментному дисплеї, по черзі запалюючи кожен розряд. Через специфічне розведення ліній керування дисплеєм у схемі, класичну таблицю кодування символів було програмно перераховано під нове розташування пінів. У результаті моделювання було отримано стабільну синусоїду на осцилографі, частота якої миттєво і точно реагувала на натискання кнопок користувача зі зміною значень на цифровому індикаторі.



Висновок. Під час виконання лабораторної роботи було успішно досліджено принципи цифро-аналогового перетворення та спроєктовано кілька діючих моделей генераторів сигналів у середовищі SimulIDE. На першому етапі практично закріплено розуміння фізичних процесів поділу та підсумовування напруг за допомогою статичної резисторної матриці R-2R. Наступним кроком стала реалізація базового програмно-апаратного ЦАП на основі мікроконтролера ATmega8, де керуюча програма мовою Асемблер забезпечила високошвидкісну генерацію ступінчастої синусоїди з використанням табличних значень. Головним досягненням роботи стала розробка комплексної системи прямого цифрового синтезу (DDS) на базі ATmega32, яка стала повноцінною діючою альтернативою запропонованій у методичних вказівках схемі з ARM-процесором LPC2138. Написання керуючого С-коду для цієї моделі дозволило глибоко опанувати роботу з апаратними таймерами, обробку зовнішніх переривань від кнопок та алгоритми динамічної індикації на мультиплексованому семисегментному дисплеї. У результаті було створено професійний генератор аналогового сигналу з можливістю точного інтерактивного керування частотою, що повністю підтверджує успішне виконання всіх поставлених завдань.

Контрольні запитання.

1. **Наведіть та поясніть формули розрахунку напруги на виході ЦАП, та коефіцієнту передачі.**
   Напруга на виході ЦАП розраховується за формулою $U = \frac{U}{2^n}\sum_{i=0}^{n-1} a_i 2^i$, де U – опорна напруга, n – кількість розрядів, а a_i – значення і-го розряду двійкового коду. Коефіцієнт передачі визначається як $K = \frac{U}{2^n}$ і показує ціну молодшого значущого розряду.
2. **Чому дорівнює максимальне значення напруги на виході ЦАП, коли у всіх розрядах вхідного коду присутні одиниці?**
   Максимальне значення напруги визначається виразом $U_{.max} = U \cdot \frac{2^n-1}{2^n}$.
3. **Чому дорівнює коефіцієнт передачі операційного підсилювача у схемі ЦАП з підсумовуванням напруг?**
   Оскільки операційний підсилювач використовується у зворотному включенні як повторювач напруги, його коефіцієнт передачі дорівнює 1.
4. **Яку роздільну здатність має ЦАП у складі мікроконтролерів ХМеда?**
   Модуль ЦАП у мікроконтролерах сімейства XMEGA має 12-розрядну роздільну здатність.
5. **Які джерела опорної напруги можуть бути використані для модуля**

ЦАП?

Можуть бути використані внутрішнє джерело на 1.00 В, зовнішня напруга живлення AV_{CC}, або зовнішня опорна напруга, що подається на спеціальні виводи AREF.

6. **Наведіть вираз, що пов'язує вихідну напругу ЦАП зі значенням, яке записано у регістр даних.**

   Цей зв'язок описується формулою $U = \frac{CHnDATA}{4096} \cdot U_{ref}$, де CHnDATA є десятковим еквівалентом числа в регістрі даних.

7. **Де можна використати аналоговий сигнал, отриманий у результаті перетворення?**

   Цей сигнал можна вивести на зовнішні виводи мікросхеми у вигляді неперервного сигналу, або ж направити всередині кристала на входи аналогового компаратора та АЦП.

8. **Для чого може бути використаний режим запуску мікроконтролера за настанням події?**

   Система подій дозволяє периферійним модулям взаємодіяти між собою без втручання центрального процесора, наприклад, ініціювати чергове перетворення ЦАП строго за переповненням таймера.

9. **Завдяки якому пристрою забезпечується двоканальне функціонування ЦАП?**

   Двоканальна робота реалізується завдяки наявності двох незалежних схем вибірки-зберігання, які підтримують рівень сигналу на кожному каналі між перетвореннями.

10. **Які часові обмеження накладаються на роботу ЦАП?**

    Інтервал вибірки та перетворення повинен складати не менше 1 мкс для одноканального режиму і 1.5 мкс для двоканального, при цьому час оновлення між каналами не повинен перевищувати 30 мкс для уникнення втрати амплітуди.

11. **Як досягається режим енергозберігання модулю ЦАП і які наслідки має використання цього режиму?**

    Режим досягається шляхом повного відключення модуля між перетвореннями для зниження споживаного струму, наслідком чого є збільшення загального часу перетворення при кожному новому запуску.

12. **Навіщо і як виконується калібрування ЦАП?**

    Калібрування необхідне для компенсації апаратних похибок підсилення та зміщення сигналу, що забезпечує високу точність перетворення. Вико-

нується воно шляхом запису розрахованих значень у спеціальні регістри GAINCAL та OFFSETCAL.

Лістинг.

firmware-mega8.asm

```
.nolist
.include "m8def.inc"
.list

.def Counter = R16
.def Temp = R17

.CSEG
.ORG 0x0000
RJMP Reset

Reset:
; Initialize stack pointer
LDI Temp, low(RAMEND)
OUT SPL, Temp
LDI Temp, high(RAMEND)
OUT SPH, Temp

Init:
; Configure Port D as output
LDI Temp, 0xFF
OUT DDRD, Temp
OUT PORTD, Temp

; Clear counter
CLR Counter

Loop:
; Load base address of sine table into Z register
LDI ZL, low(Sinus*2)
LDI ZH, high(Sinus*2)

; Add current counter value to address
ADD ZL, Counter
CLR Temp
ADC ZH, Temp

; Read sine value from flash memory
LPM Temp, Z

; Output value to DAC (Port D)
OUT PORTD, Temp

; Increment counter (auto-wraps to 0 after 255)
INC Counter

; Repeat loop
RJMP Loop

; Sine wave values table (256 values)
Sinus:
.DB 64,65,67,68,70,72,73,75
.DB 76,78,79,81,82,84,85,87
.DB 88,90,91,92,94,95,97,98
.DB 99,100,102,103,104,105,107,108
.DB 109,110,111,112,113,114,115,116
.DB 117,118,118,119,120,121,121,122
.DB 123,123,124,124,125,125,126,126
```

```
.DB 126,127,127,127,127,127,127,127
.DB 128,127,127,127,127,127,127,127
.DB 126,126,126,125,125,124,124,123
.DB 123,122,121,121,120,119,118,118
.DB 117,116,115,114,113,112,111,110
.DB 109,108,107,105,104,103,102,100
.DB 99,98,97,95,94,92,91,90
.DB 88,87,85,84,82,81,79,78
.DB 76,75,73,72,70,68,67,65
.DB 64,62,61,59,58,56,54,53
.DB 51,50,48,47,45,44,42,41
.DB 39,38,36,35,34,32,31,30
.DB 28,27,26,25,23,22,21,20
.DB 19,18,17,15,14,13,13,12
.DB 11,10,9,8,8,7,6,5
.DB 5,4,4,3,3,2,2,2
.DB 1,1,1,0,0,0,0,0
.DB 0,0,0,0,0,0,1,1
.DB 1,2,2,2,3,3,4,4
.DB 5,5,6,7,8,8,9,10
.DB 11,12,13,13,14,15,17,18
.DB 19,20,21,22,23,25,26,27
.DB 28,30,31,32,34,35,36,38
.DB 39,41,42,44,45,47,48,50
.DB 51,53,54,56,58,59,61,62
```

firmware-mega32.c

```c
#include <avr/io.h>
#include <avr/interrupt.h>

// Sine wave values table (256 values centered at 128)
const unsigned char SINE_TABLE[256] = {
    64,65,67,68,70,72,73,75,76,78,79,81,82,84,85,87,
    88,90,91,92,94,95,97,98,99,100,102,103,104,105,107,108,
    109,110,111,112,113,114,115,116,117,118,118,119,120,121,121,122,
    123,123,124,124,125,125,126,126,126,127,127,127,127,127,127,127,
    128,127,127,127,127,127,127,127,126,126,126,125,125,124,124,123,
    123,122,121,121,120,119,118,118,117,116,115,114,113,112,111,110,
    109,108,107,105,104,103,102,100,99,98,97,95,94,92,91,90,
    88,87,85,84,82,81,79,78,76,75,73,72,70,68,67,65,
    64,62,61,59,58,56,54,53,51,50,48,47,45,44,42,41,
    39,38,36,35,34,32,31,30,28,27,26,25,23,22,21,20,
    19,18,17,15,14,13,13,12,11,10,9,8,8,7,6,5,
    5,4,4,3,3,2,2,2,1,1,1,0,0,0,0,0,
    0,0,0,0,0,0,1,1,1,2,2,2,3,3,4,4,
    5,5,6,7,8,8,9,10,11,12,13,13,14,15,17,18,
    19,20,21,22,23,25,26,27,28,30,31,32,34,35,36,38,
    39,41,42,44,45,47,48,50,51,53,54,56,58,59,61,62
};

// Seven segment digit codes (Custom wiring: PC7=a, PC6=b, PC5=c, PC4=d, PC3=e, PC2=f, PC1=g)
const unsigned char SEGMENT_CODE[10] = {
    0xFC, // 0
    0x60, // 1
    0xDA, // 2
    0xF2, // 3
    0x66, // 4
    0xB6, // 5
    0xBE, // 6
    0xE0, // 7
    0xFE, // 8
```

```c
    0xF6  // 9
};

// Masks for selecting digits on PORTD (PD7 is leftmost digit, PD4 is rightmost)
const unsigned char DIGIT_SELECT[4] = {0x70, 0xB0, 0xD0, 0xE0};

volatile unsigned int current_frequency = 500;
volatile unsigned int phase_accumulator = 0;
volatile unsigned int tuning_word = 0;

volatile unsigned char display_buffer[4] = {0, 0, 0, 0};
volatile unsigned char active_digit = 0;

// Update tuning word and display buffer based on new frequency
void set_frequency(unsigned int freq)
{
    // Limit frequency boundaries
    if (freq > 9999) freq = 9999;
    if (freq < 10) freq = 10;

    current_frequency = freq;

    // Calculate DDS tuning word: (freq * 2^16) / sample_rate
    // Sample rate = 8000000 / 256 = 31250 Hz. Constant is roughly 2.097
    tuning_word = (current_frequency * 2147UL) >> 10;

    // Update display digits
    display_buffer[0] = SEGMENT_CODE[current_frequency / 1000];
    display_buffer[1] = SEGMENT_CODE[(current_frequency / 100) % 10];
    display_buffer[2] = SEGMENT_CODE[(current_frequency / 10) % 10];
    display_buffer[3] = SEGMENT_CODE[current_frequency % 10];

    // Blank the leading zero if frequency is below 1000
    if (current_frequency < 1000) {
        display_buffer[0] = 0x00;
    }
}

// Timer0 Overflow Interrupt: Sine wave generation (Runs at ~31.25 kHz)
ISR(TIMER0_OVF_vect)
{
    phase_accumulator += tuning_word;
    PORTA = SINE_TABLE[phase_accumulator >> 8];
}

// Timer1 Compare Match Interrupt: Display multiplexing (Runs at 1 kHz)
ISR(TIMER1_COMPA_vect)
{
    // Turn off all digits to prevent ghosting
    PORTD |= 0xF0;

    // Output segment data
    PORTC = display_buffer[active_digit];

    // Turn on the active digit while preserving other bits
    PORTD = (PORTD & 0x0F) | DIGIT_SELECT[active_digit];

    // Move to the next digit
    active_digit++;
    if (active_digit > 3) active_digit = 0;
}

// External Interrupt 0: Increase frequency
```

```c
ISR(INT0_vect)
{
    set_frequency(current_frequency + 50);
}

// External Interrupt 1: Decrease frequency
ISR(INT1_vect)
{
    set_frequency(current_frequency - 50);
}

int main()
{
    // Configure Ports
    DDRA = 0xFF;  // PORTA as output for DAC
    DDRC = 0xFF;  // PORTC as output for 7-segment data
    DDRD = 0xF0;  // PD4-PD7 output for digit selection, PD2-PD3 input for buttons

    // Enable pull-up resistors on PD2, PD3 and disable all digits initially
    PORTD = 0xFC;

    // Configure Timer0 for DDS generation (Normal mode, No prescaler)
    TCCR0 = (1 << CS00);
    TIMSK |= (1 << TOIE0);

    // Configure Timer1 for display multiplexing (CTC mode, Prescaler 64)
    TCCR1B = (1 << WGM12) | (1 << CS11) | (1 << CS10);
    OCR1A = 124; // 8MHz / 64 / 125 = 1000 Hz refresh rate
    TIMSK |= (1 << OCIE1A);

    // Configure External Interrupts for buttons (Trigger on falling edge)
    MCUCR = (1 << ISC11) | (1 << ISC01);
    GICR = (1 << INT1) | (1 << INT0);

    // Initialize starting frequency
    set_frequency(500);

    // Enable global interrupts
    sei();

    // Infinite loop, everything is interrupt-driven
    while(1) {
    }

    return 0;
}
```